

Device Type Classification Using WiFi Probe Requests: From Signals to Insights

Niruth Bogahawatta,* Yasiru Senarath Karunanayaka ,* Suranga Seneviratne,* Kanchana Thilakarathna,*

Rahat Masood,[†] Salil S. Kanhere,[†] Aruna Seneviratne [†] Albert Zomaya *

* University of Sydney, Australia, [†] University of New South Wales, Australia,

Email: {firstname.lastname}@sydney.edu.au, {firstname.lastname}@unsw.edu.au

Abstract—WiFi devices are ubiquitous in modern environments, from smartphones and laptops to IoT sensors and AR/VR headsets. Identifying device types/models within these populations enables crowd analysis, network optimization, and detection of unusual devices. Current identification methods struggle with MAC address randomization, require large training datasets, and perform poorly in real-world deployments. This paper introduces a device identification method based on Information Element (IE) attributes extracted from WiFi probe requests. We evaluate the approach using probe requests captured in the 2.4 GHz band. Evaluation across 70+ device types yields 99% precision, 98% recall, and 99% F1 score, exceeding deep learning approaches (92% F1 score) under similar training conditions. Our approach maintains accuracy despite MAC randomization and requires minimal training data. We demonstrate practical applicability through an operational dashboard tested in real-world scenarios for urban planning and network management. Case studies across diverse environments confirm the effectiveness of the method for operational use.

Index Terms—WiFi, Device Identification, WLAN, Wireless Networks

I. INTRODUCTION

WiFi has become ubiquitous, transforming how we live, work, and interact with the Internet. Its wide adoption stems from its compatibility with a diverse range of devices, including smartphones, laptops, tablets, smart TVs, and gaming consoles. The rise of smart homes and the Internet of Things (IoT) further expands WiFi's reach, linking devices like thermostats, security cameras, and voice assistants into a unified digital ecosystem. Its ubiquity enables intelligence gathering that can support public safety, law enforcement, and urban design [26], [31], [32].

This extensive adoption of WiFi also enables passive identification of WiFi devices based on the signals they emit, such as probe requests, without requiring active communication or cooperation. This enables scalable, non-intrusive identification while respecting privacy. Passive WiFi identification can be used in various applications, leveraging existing WiFi infrastructure to gather intelligence. For example, passive WiFi identification can help urban planners identify areas with differing levels of technological engagement. High-tech neighborhoods might benefit from smart city solutions, while connectivity enhancements can be prioritized in low-tech areas to bridge the digital divide. By avoiding external cooperation or active communication, passive methods scale better and reduce detection risk for urban and security applications

Early work in passive WiFi device identification [18], [37] relied on WiFi sensing, which involves detecting and analysing wireless frames generated by WiFi-enabled devices without direct interaction. These methods typically relied on MAC addresses for device and device type identification. Such methods became less effective once MAC address randomisation was deployed [20]. Subsequent work exploited the fact that WiFi devices routinely emanate probe requests and analysed the probing behaviours of devices for identification. Here, the focus was on the transmission pattern of probe requests [12], [21], [36]. Works such as [4], [7] focus on device fingerprinting (DFP) using the Inter Arrival Time (IAT) of probe requests where graphs were created for the IAT of probes and fed into a CNN (Convolution Neural Networks) for classification. However, when measuring IAT, data must be collected continuously without experiencing packet drops, ensuring a seamless, uninterrupted stream of data. This poses a challenge as the entity responsible for passive data collection does not have control over the data flow, as they don't actively participate in the network. Research like [23] uses neural networks, and [24], [34] uses a mix of unsupervised and supervised learning to classify mobile devices based on probe request Information Elements (IEs) (cf III-A). However, these methods require numerous packets per device for training, necessitating a labor-intensive dataset collection process. Their effectiveness tends to be limited to specific datasets and exhibits diminished accuracy when scaled across diverse datasets [40]. A recent approach, BLEACH [29], identifies unique device signatures from probe request transmissions using a combination of signature-based methods and conflict-resolution techniques. However, it does not account for the variability in the generated signatures due to changes in the Information Elements associated with a given device or device type over time. These changes could impact the stability of the generated signatures and affect the long-term effectiveness of the method.

In contrast, this paper presents a passive method for identifying WiFi device types using Information Elements (IEs) extracted from Probe Request frames. We focus on undirected probe requests with a wildcard SSID, i.e., active scan frames not targeted to a specific AP [22]. In our evaluation, we use probe requests captured in the 2.4GHz band to construct persistent device-type signatures. Our approach does not assume a single fixed probe signature; instead, it explicitly models within-device variability by retaining multiple observed values

per field and weighting IEs by their prevalence across the population (cf. Section III-A3). These signatures resist MAC address randomization, ensuring stable identification. Our objective is not to identify individual devices, but to distinguish device types at fine granularity, e.g., differentiating models like the iPhone 13 and 14, or Pixel 6 and 6a. For classification, we compute a similarity metric using a weighted average probability reflecting each IE's uniqueness. This enhances differentiation by considering the global distribution of each IE. The main contributions of this paper are as follows:

- We developed a data-driven approach to build device-type signatures from the probability of IE occurrence in probe requests. Unlike existing machine learning methods, our approach generates signatures from only a few WiFi probe request observations.
- We show our signature-based solution achieves an average F1 of 99%, precision 99%, and recall 98% for device-type identification. It consistently surpasses deep learning such as [23], reaching higher accuracy overall with far less training data—achieving a 92% F1 with just one training example per device type.
- We curated an extended dataset of 73 device types, building on our previous work [9], to increase signature diversity and better reflect real-world distributions, including a broader range of IoT devices. Additionally, we performed a temporal analysis empirically showing signature stability over time, validating their reliability for long-term use.
- We develop a custom dashboard implementing our signature-based device identification and conduct a pilot study clearly demonstrating applicability in real-world scenarios. The study details potential use cases in depth, showcasing the method's effectiveness and practical value.

The remainder of this paper is as follows. In Section II, we present the related work. We describe our signature creation and matching methodology in Section III and our data collection process in Section IV. We present our results and analysis in Section V. Finally, Section VIII concludes the paper.

II. RELATED WORK

There are several passive device identification problems associated with WiFi. Our work addresses WiFi device type identification, aiming to identify the make and model of WiFi devices through passive observation. Another related challenge is WiFi device re-identification, which involves tracking the reappearance of the same device across different WiFi networks over time and space. Existing work utilize either MAC layer (Layer 2) or Network Layer (Layer 3) information, depending on the scenario and available data assumptions. Network layer information, containing device-specific details like device names, is effective but requires network access, including the network password. To circumvent this limitation, researchers focused on using MAC layer information, which can be collected without connecting to the WiFi network. This approach allows device identification without needing network access. This aligns with the principle of data minimization, as

emphasized in Cavoukian's privacy-by-design framework [10], and supported in related work on WiFi privacy and device fingerprinting [38], [39], which highlight the importance of limiting data collection to only what is relevant and necessary for the intended task.

WiFi MAC addresses, comprised of 12 hexadecimal digits, serve as unique device hardware signatures, and the vendor-specific first 24 bits of the MAC address allowed for device type identification. Privacy concerns regarding static MAC addresses were highlighted in multiple studies [14]–[16], leading manufacturers to implement MAC address randomisation, which conceals the original MAC address during communication and complicates device identification [28]. MAC randomisation can be triggered by events such as device lock/unlock, WiFi interface activation/deactivation, or connection attempts to a WiFi access point. A randomised MAC address adheres to the standard format but can be identified by checking if the second least significant bit of the first byte is set to 1 [13], i.e., the second character of the MAC address is 2, 6, A, or E.

Early MAC address randomisation had deployment issues, with inconsistent usage across devices [35], still allowing the possibility of both device and device type identification. For example, initially, 96% of Android devices failed to effectively hide their MAC addresses, occasionally reverting to broadcasting the original MAC. In addition to deployment inconsistencies, management frames contained persistent identifiers like Universally Unique Identifier (UUID-E), which remains static even when the MAC address changes. Attackers could exploit UUID-E to recover the device's MAC address. The WiFi frames also included a 12-bit sequence number, facilitating the linking of random MAC address changes due to the sequential incrementing of the field [28]. Since then, these limitations have been addressed. Most devices released in the last few years do not include the UUID-E field and use randomised sequence numbers, including Android devices [20]. As a result, subsequent work focused on device and device type identification under the challenging conditions of randomised MAC addresses.

Works such as [24], [34] highlight the impact of MAC address randomization on statistical applications such as people counting, crowd flow estimation, and trajectory inference. Despite these challenges, the study demonstrates that frames from a single device can still be associated even with varying random MAC addresses using features like probe requests. Probe requests, operating at the OSI model's data link layer (Layer 2), are management frames broadcast by devices when searching for networks. Espresso [34] introduces a framework for associating WiFi probe requests under MAC address randomization. Espresso follows a two-step process: first, it calculates association probabilities between frame pairs using a multimodal correlation estimator integrating Information Elements (IEs), sequence numbers, and RSSI. Second, it performs frame association by modeling frames as nodes in a flow network, resolving conflicts via minimum-cost flow optimization. However, the framework's reliance on RSSI, which can be unstable in real-world conditions, and limited testing across diverse scenarios restrict its broader applicability. Building on Espresso, Cappuccino [24] introduces a self-

supervised frame correlation estimator using a Siamese neural network and contrastive learning, reducing manual labeling needs.

Additionally, [18] proposed tracking and re-identifying WiFi devices across different networks using the preferred network list (PNL). The PNL is a historical record of all network names (SSIDs) a device has previously connected to. However, this approach is no longer applicable since probe requests no longer advertise the PNL.

In the context of device type identification, several works [12], [21], [36] delved into the probing behavior of mobile devices, focusing on the patterns of probe request transmissions. This includes the number of probe requests sent in specific time frames and the number of consecutive frames sent in one session. Works such as [4], [7] concentrate on device fingerprinting utilising the IAT. This entails feeding the plotted graphs based on IAT into machine learning models such as CNNs and Long Short-Term Memory (LSTM) networks. However, a significant challenge with these methods is the requirement for a continuous, uninterrupted flow of data, which passive data collection struggles to provide. Moreover, the frequency and quantity of probe requests can vary based on the device's operating mode, as observed in studies such as [17]. For instance, the number of probes sent during power-saving mode may differ from regular usage, impacting the method's accuracy in correctly identifying devices. Gu et al. [23] propose a device identification method that extracts features from probe request frames, focusing on the VHT capabilities IE. Using a Multi-Layer Perceptron (MLP) model, the method achieves high accuracy in classifying devices from a dataset of 20 device types. While the approach shows strong accuracy, relying on a single IE, such as the VHT capabilities, is limiting, as not all devices include this IE. Given the evolving nature of Wi-Fi standards, device identification should incorporate a combination of IEs to improve reliability and account for a wider range of devices. Further, training the MLP model requires large datasets, complicating deployment in varied environments. A more recent approach focuses on identifying unique signatures from probe request transmissions to differentiate devices. BLEACH [29] combines signature-based and conflict-resolution methods. It extracts key features from probe request frames, such as information elements (IE), sequence numbers (SEQ), temporal patterns, and RSSI values. These signatures are then used within a conflict-resolution framework, modeling MAC address conflicts over short time intervals. The process involves extracting MAC address trails, identifying conflict periods, and resolving randomized MACs with a logistic regression model trained on the extracted features. A limitation of BLEACH is its reliance on the 2013 Sapienza dataset [8], which does not reflect recent changes in wireless standards and device behaviors, such as increased IE variability in newer devices and randomized sequence numbers in Apple devices. This limits its applicability to modern devices and evolving randomization techniques. In our recent work [9] we propose a signature-based approach for device type identification, utilizing the IEs from WiFi probe requests to create device type signatures. This method allows for signature generation from a single probe request

sample and the ability to dynamically update these signatures as additional variations are observed.

This work builds on that foundation by introducing a new signature creation method that enhances the distinction between device types through more granular information. We also validate the robustness of these signatures by demonstrating their time invariance. To further support our approach, we have developed a dashboard for real-time monitoring, showcasing the applicability of our method in real-world scenarios.

III. METHODOLOGY

Our main objective is to generate a unique signature for passive identification of various WiFi device types in a specific location. To be effective, this signature must meet several key requirements:

- **Passive measurements** - the device type signature must be derived from information that can be collected passively without interacting with the device. Active sensing requires additional information like network passwords or network administrator cooperation and is not scalable (see Section I).
- **Uniqueness** - The signature must uniquely identify a specific device type. The signature should differentiate between Google Pixel 5 and 6 phones, but it should be similar for two Google Pixel 5 devices.
- **Robustness** - For a given device type, the signature must be robust to within-device variability in probe request IEs arising from operating conditions (e.g., power management and usage), and across different wireless environments and channels.

Next, we describe our signature creation process and how it achieves the above design goals.

A. Signature Creation

1) *Probe requests*: We construct device type signatures using *WiFi Probe Requests*, which are WiFi Management Frames used by devices to scan for nearby networks. Probe requests can be directed (seeking a specific WiFi SSID) or undirected (with a wildcard SSID). Capturing probe requests is straightforward using standard WiFi cards in monitor mode, allowing passive capture without a network connection.

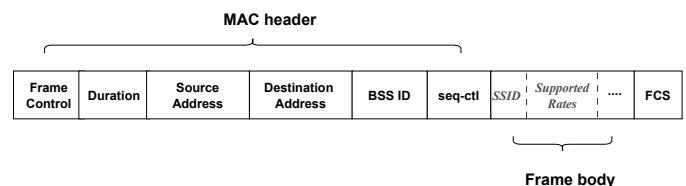


Fig. 1: The structure of a Probe Request Frame

A probe request frame contains information elements, which are data structures with numerical labels describing the device's capabilities. These elements enable the differentiation of devices connecting to a WiFi network (i.e., achieving uniqueness and robustness). We show the structure of a probe request in Figure 1, and the most relevant part to our signature building process is the *Frame Body*.

2) *Information Elements (IE) of probe requests*: Undirected probe requests have a null value in the SSID field. The rest of the frame body contains IEs containing information about the WiFi device, such as supported data rates, security protocols, and device capabilities. It is worth noting that not all IE's will be present within a probe request frame body. Devices exhibit diverse capabilities, and the absence of specific IEs in a Probe Request can indicate a device's lack of support for specific WiFi standards, encryption methods, or other optional features. Therefore, the contents of IEs and the presence or absence of an IE provide valuable insights into the characteristics of a WiFi device. Table I outlines available IEs and the scenarios they will be present within the frame body according to the IEEE standard [33].

Across our data, the most predominant such attributes were Supported rates and Extended Supported Rates, High Throughput (HT) and Very High Throughput (VHT) capabilities, as well as Extended and High Efficiency (HE) capabilities. Below, we provide a more detailed description of these prominent IEs.

TABLE I: IEs within the frame body of a probe request

Information Elements	Description
Supported Rates	Data rates that the Wi-Fi device supports for communication
Extended Supported Rates	Additional supported rates beyond the initial eight rates. Present if more than eight data rates are supported
Operating Classes	Indicates the supported frequency channels and operating classes of the Wi-Fi device. Present if channel switching is activated
High Throughput	High Throughput capabilities. Present if the device supports high throughput options
Very High Throughput	Very High Throughput capabilities. Present if the device supports 802.11ac
Interworkings	Information related to interworking services to facilitate seamless communication. Present if interworking services are activated
Mesh ID	Mesh networking capabilities of the device, allowing it to participate in wireless mesh networks. Present if mesh networking is activated
Vendor-Specific	One or more elements providing vendor-specific information. Optionally present to offer detailed vendor-specific data
High Efficiency	High-efficiency capabilities introduced in 802.11ax. Present if Wi-Fi 6 is supported
Extended	Information related to supported 802.11 amendments

- **Supported & Extended Supported Rates** — WLANs standardize data rates. The Supported Rates IE (ID 1) lets a network learn which legacy PHY rates a device supports; on join, the device verifies the network's rates and which are mandatory or optional. In IEEE 802.11, this IE is a byte string whose octets each carry one rate: the MSB flags a basic/mandatory rate, the remaining seven bits encode the rate in 0.5 Mb/s units (e.g., 2 → 1 Mb/s; 108 → 54 Mb/s), with up to eight rates per element. If more than eight are advertised, the Extended

Supported Rates IE (ID 50) continues the list with the same per-octet format. As with all IEs, the Length field bounds payload size, and the element appears only when additional legacy rates must be listed.

- **High Throughput (HT) and Very High Throughput (VHT)** — HT (IE ID 45, IEEE 802.11n [2]) advertises 11n features such as 40MHz channels, short guard interval, and MIMO. The HT Capabilities IE has a fixed layout: HT Capabilities Info (bitmask of supported features), A-MPDU Parameters, Supported MCS Set (bitmap listing supported MCS indices per spatial stream), HT Extended Capabilities, and optional Transmit Beamforming. VHT (IE ID 191, IEEE 802.11ac [3]) advertises 11ac features including 80/160MHz operation and higher-order MCS. The VHT Capabilities IE contains a VHT Capabilities Info bitmask and a VHT Supported MCS Set; the latter encodes per-spatial-stream RX/TX support using codes indicating “not supported” or MCS ranges (e.g., 0–7, 0–8, 0–9), and may include the max data rate. These capability IEs appear only when the device implements the corresponding PHY.
- **Extended** — The Extended (IE ID 127) element carries device details or vendor/amendment information that doesn't fit elsewhere; contents may indicate supported 802.11 amendments, channel-width capabilities, or other feature flags. Per the 802.11 IE format, it uses the Element ID, Length, Information triplet; the Information field is variable and can aggregate multiple capability indicators, so length varies by device and firmware. In practice, it appears only when the device has such extended indications to advertise.
- **High Efficiency (HE)** - HE (ID 255) is defined in IEEE 802.11ax (Wi-Fi 6) [27] and improves spectral efficiency and multi-user performance through features such as OFDMA, BSS Coloring, Target Wake Time (TWT), and spatial reuse. In the standard, the HE Capabilities element encodes these features in a variable-length structure composed of HE MAC/PHY capability bitmasks and an HE MCS/NSS set that advertises per-stream MCS support; the element's length and subfields depend on which 11ax features the device implements, and it is included only when HE is supported.

3) *Signature*: After capturing probe requests(see Section IV-A), we derive device type signatures by extracting all Information Elements (IEs) from each request (Algorithm 1). Example signatures for a Samsung Galaxy A13 and a Google Pixel 7 appear in Figure 2. Each signature is a dictionary mapping IE IDs to values; because these are key value pairs, we treat the signature as an unordered set. Thus, the order of IEs in a probe request has no effect, since each IE is treated as an independent entry.

In Algorithm 1, we deliberately skip certain IE IDs, specifically {0, 3, 255}, because they carry no stable, discriminative information across devices. ID 0 corresponds to the SSID element. Because we restrict analysis to wildcard probe requests, the SSID is empty and provides no discriminative information for device-type identification; in directed probes it

$$T(p) = \begin{cases} T_1(p) = \frac{1}{\alpha_1 - \beta_1(1-p)} - \frac{1}{\alpha_1} \\ \alpha_1 = 1 - \frac{2}{\lambda} \quad \beta_1 = \frac{\lambda^2 - 4\lambda + 4}{\lambda^2 - \lambda} \\ T_2(p) = \alpha_2 e^{\beta_2(1-x)-1} \\ \alpha_2 = \frac{\lambda}{e^{\beta_2}-1} \quad \beta_2 = 2 \ln \frac{\lambda + \sqrt{\lambda^2 - 4\lambda + 4}}{2} \\ T_3(p) = \alpha_3(1-x)^{\beta_3} \\ \alpha_3 = \lambda \quad \beta_3 = \frac{\ln \lambda}{\ln 2} \end{cases} \quad (4)$$

Figure 5 compares three transformation functions, T_1 , T_2 , and T_3 , across four values of $\lambda \in \{2.1, 2.5, 5, 10\}$. All three functions satisfy the constraints $T(0) = \lambda$ and $T(0.5) = 1$, but differ in how they weight rare probabilities. Among them, only T_1 remains well defined throughout the domain $p \in [0, 1]$ for all $\lambda > 2$. When $\lambda = 2.1$, $T_1(p)$ exhibits the steepest decline as $p \rightarrow 0$, providing strong separation between extremely rare and moderately rare events.

TABLE IV: F_1 scores for λ values using T_1 .

	2.1	2.5	5.0	10.0
F1 Score	0.99	0.99	0.97	0.96

We select $\lambda = 2.1$ for T_1 based on train-test validation. A grid search over candidate λ values shows that $\lambda = 2.1$ achieves the highest F_1 score on a held-out test set (Table IV). While $\lambda = 2.5$ yields similar performance, we use $\lambda = 2.1$ as it is the first usable value in the range. Therefore, all experiments use T_1 with $\lambda = 2.1$, producing the weights reported in Table V for the attribute similarity between signA and signB. Other values from T_2 , T_3 , or alternative λ settings remain valid options depending on the specific application.

TABLE V: Weights based on the probability of occurrence

Attribute	similarity(s_i)	probability(p_i)	weight(γ_i)
supported rates	0.555	0.05	1.981
ht	0.884	0.25	1.536
extended supported rates	1.000	0.08	1.916
extended	0.000	0.5	1.000

Next, we calculate the weighted average similarity using (5) based on the provided similarity values and corresponding attribute weights.

$$S = \frac{\sum_{i=0}^N s_i \gamma_i}{\sum_{i=0}^N \gamma_i} \quad (5)$$

The range of S will fall between 1 and 0. s_i is the similarity between signA and signB in the i^{th} attribute and γ_i is the weight for the i^{th} attribute of both signatures as calculated by (4). For instance, substituting (5) with the values from Table V, we obtain the weighted average similarity between signA and signB as 0.68.

3) *Handling different combinations of IE values:* When creating a signature for a particular device, probe requests often show varied attribute combinations due to different IE values or varying numbers of IEs across requests. For instance, in signA, extended supported rates may include either [48,

72, 96, 108] or [18, 48, 72, 96, 108], with frequencies that are not uniform. Our earlier calculations failed to account for this variability. The probability of occurrence we previously calculated was based on the likelihood of encountering a specific attribute value within the dataset. Therefore, we computed the probability of an attribute/attribute value being present for each query signature. These values were derived by analysing all the probe requests linked to that particular device.

TABLE VI: Likelihood of an attribute value occurring for Extended Supported rates and Supported rates for signA

Attribute	Attribute value	P(signA)
extended supported rates	48, 72, 96, 108	0.78
	18, 48, 72, 96, 10	0.22
supported rates	2, 4, 11, 12, 18, 22, 24, 36	0.91

Table VI illustrates occurrence probabilities for signA. Two potential values for extended supported rates indicate signA can have two distinct attribute values, each with its weighting. The presence of one value with a likelihood of 0.91 for supported rates suggests that 9% of signA probe requests lacked a supported rate value, reflecting variances in the number of Information Elements (IEs) across requests. The amended calculation becomes:

Substituting the values from Table VI into $S = \frac{\sum_{i=0}^N s_i \gamma_i l_i}{\sum_{i=0}^N \gamma_i l_i}$ where l_i is the occurrence probability, yields a new weighted average similarity of 0.66 between signA and signB. The decrease in similarity score from 0.68 to 0.66 between signA and signB in the revised calculation is due to the adjustment of weights based on the probability of each attribute/attribute value being present within the query signatures. This adjustment ensures that an IE combination occurring less frequently will be penalised less for a given device, resulting in a higher similarity score (due to the weight for that attribute being scaled by its occurrence). Conversely, for two different devices, the similarity score will be lower. This adjustment effectively accounts for the varying occurrence frequencies of different IE combinations and helps provide more accurate similarity assessments between query and target signature.

C. Feature Selection

We evaluated the predictive power of various Information Elements (IEs) for identifying specific device types by calculating the Gini importance using a random forest classifier, as depicted in Fig. 6. Key elements like “ht,” “extended,” and “vendor-specific” information emerged as critical for device type identification.

Relying solely on the most important features may not be practical because they might not be present on all devices. Therefore, we adopt a two-phase approach to assess similarity when comparing two signatures. When evaluating the similarity between signA and signB, we initially focus on their top three shared features, such as “HT”. Then, assess all features, including “HT”, supported rates, and extended supported rates. This approach yields two separate scores. The final score combines these evaluations using a weighted ratio of 9:1. This ratio, determined through iterative experimentation,

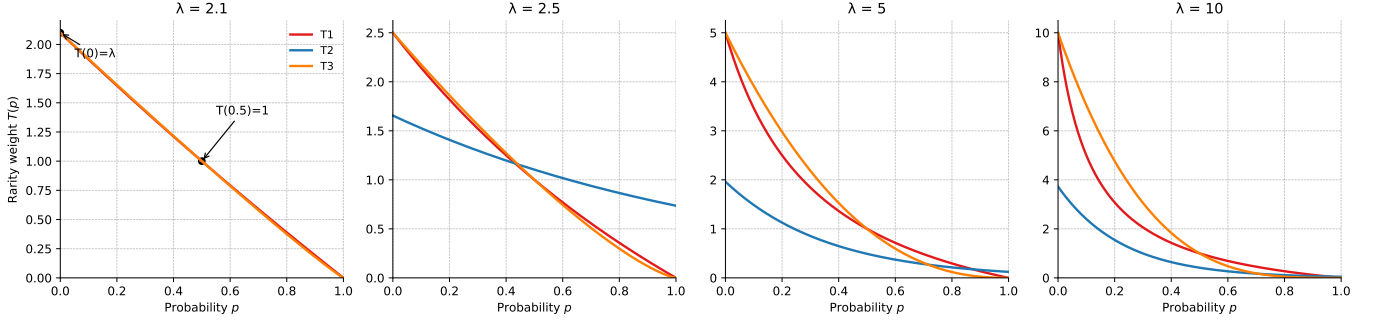


Fig. 5: Hyper parameter tuning for λ

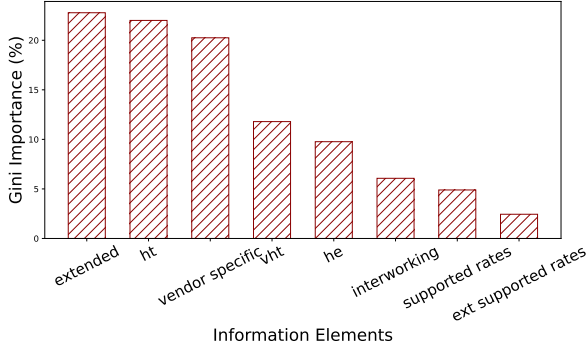


Fig. 6: Gini Importance of features

optimizes our comparison results. In practical terms, the similarity score between signA and signB can be computed as $0.84 \times 0.9 + 0.66 \times 0.1 = 0.81$. This includes an initial assessment focusing on critical features and a subsequent comprehensive evaluation of all available features.

IV. DATA

A. Data Collection & Setup

We collected data in a university lab after approval from the Human Research Ethics Committee (HREC). Our objective was to capture Wi-Fi probe request frames—management frames broadcast by client devices during network discovery to identify nearby Access Points (APs). The hardware setup comprised three ALFA AWUS036NHA external Wi-Fi adapters connected either to a Dell XPS 15 running Linux or to a Raspberry Pi running Raspbian. Using custom Python scripts controlling *tshark* [1], we recorded traffic in 10-minute intervals while systematically hopping across all 14 Wi-Fi channels. During each session, we invited nearby students and staff to participate. For each participant's device (e.g., smartphone, laptop, or IoT device), we recorded the make, model, and MAC address, establishing ground truth at capture time. In addition, for the following device families we manually disabled MAC randomization (enforcing fixed MACs): Samsung, Google Pixel, Apple (iPhone/iPad), Meta Quest, MacBook, ASUS ZenBook, and Raspberry Pi units. We disabled MAC randomization only during controlled sessions to associate probes with the participant device when filtering PCAPs; the method itself uses IE-based signatures, not MAC addresses. The raw data were saved as large *.pcap* files containing all

Wi-Fi traffic. Using the recorded MAC addresses, we filtered the captures to isolate probe request frames per device, producing individual *.pcap* files that contained only that device's probe requests. This process was repeated over three days (approximately four hours of capture per day), after which the per-day files were merged into a single comprehensive *.pcap* per device type. These per-device-type probe request PCAPs form the input to our supervised experiments. As described in Section V-A, for each device type we randomly split its available probe request frames into two subsets (50:50): one subset is used for training/signature construction and the other subset is held out for testing/evaluation.

B. Preprocessing Data

We filter the collected data to extract probe requests. We retained only wildcard-SSID requests and ignored directed probes. Since we focus on wildcard probe requests, these frames are not directed to any specific SSID or access point (SSID is empty) and therefore do not encode a specific network identity. We analyze only the wildcard Probe Requests and treat variability in their IE contents as part of the within-device variability captured by our signature representation. The filtered probe requests were then processed to extract specific Information Elements (IEs) from all frames for each device type. Frames containing probe requests with less than two or fewer IEs were deliberately discarded during this process due to the insufficiency of one attribute in identifying a device accurately. Additionally, we enhanced our dataset by incorporating the probe request dataset released by Pintor & Atzori [30]. This dataset consists of probe request capture files from 22 devices recorded over a 20-minute duration. Our dataset includes 73 different device types, comprising 88,668 probe request capture frames. We utilized 51,726 of these frames for analysis, discarding the remaining 36,942 frames that were not sent to the wildcard SSID.

Since we focus on wildcard Probe Requests, these frames are not directed to any specific SSID or access point (the SSID field is wildcard/empty) and therefore do not encode a specific network identity.

V. EXPERIMENTS AND RESULTS

A. Experiment setup

To evaluate our signature-based device identification, we split the dataset 50/50 per device. For each of the 73 devices,

TABLE VII: Frequency of Probe Requests per Device

Device Name	No. Probe requests	Device Name	No. Probe requests
Samsung S4	14123	Apple iPhone 11 Pro	72
Xiaomi Redmi 4	6006	MacBook Pro 2015	70
Xiaomi Redmi 5 Plus	4208	Raspberry Pi4	65
Xiaomi Redmi Note 7	3036	Samsung Tab S8	62
iPad 3rd Gen	2939	Google Pixel 2	61
Huawei P20	2628	Solar Edge	60
Xiaomi Mi A2	2309	Google Pixel 6 Pro	60
Apple iPhone 12	2237	Samsung S22	58
Huawei STF L09	1996	VIP Vision Bell	57
Huawei ALE L21	1541	Samsung A13	56
Xiaomi Redmi Note 9S	1423	Apple iPhone 15 Pro	55
Huawei VTR L09	861	Holo Lens 2	53
Apple XS	799	Eufy Cam S3 Solar	45
Apple iPhone 6	787	Asus ROG Strix G15	45
OnePlus Nord	730	Samsung A32	38
Samsung J6	699	Samsung A15	35
Samsung S7	697	Samsung A33 5G	31
Google Home 2	598	MacBook Air M1	29
Google Pixel 3a	533	Xiaomi Poco F1	27
Samsung M31	485	Eufy Home Base	26
Samsung S6 Edge Plus	459	Samsung S3	26
Apple iPhone 13 Pro	287	Samsung TV	26
Samsung A52	275	Dell Latitude 5460	19
Apple iPhone 14 Pro	207	Google Pixel 8	19
Apple iPhone XR	146	Samsung Tab A8	18
Samsung Note 20 Ultra	138	Raspberry Pi3	17
Xiaomi Mi9 Lite	135	Google Pixel 7a	13
Apple iPhone 11	129	Holman WX1 Wifi Tap	11
Samsung A73 5G	109	Asus Zenbook 14	10
Apple iPad 8	100	Meta Lens 2	9
Google Pixel 6a	95	Dell XPS 13 9380	8
Xiaomi Redmi Note 8T	89	Google Nexus 1	6
Google Pixel 5	82	Google Pixel 7	5
Apple MacBook 14 22	79	Lenovo X13 Gen1	4
Apple iPhone 7	75	TPLink Smart Globe	4
Neo Connect Control	72	TPLink Smart Plug	4

we randomly sampled probe requests for training and testing to mitigate biases like temporal dependencies.

We benchmarked our method against three representative baseline categories: traditional decision tree classifiers, the BLEACH framework [29], and a neural network model tailored to probe-request classification. Together, these baselines provide a comprehensive assessment across traditional, prior-art, and learning-based approaches.

We began with decision tree-based methods, including standalone decision trees, Random Forests, and XGBoost. These models offer a natural comparison because they also perform rule-based classification over discrete protocol features. To prepare input for these models, each probe request was first converted into a variable-length list of “IE=value” tokens. These lists were transformed into frequency dictionaries, capturing how often each token occurred within a trace. We then applied scikit-learn’s `FeatureHasher` to convert these sparse dictionaries into fixed-length vectors. The hashing dimension D was selected via grid search from the range 512 to 8192. This technique avoids padding or truncation while preserving the uniqueness of tokens. Labels were one-hot encoded, and after tuning on an 80/20 train-validation split, each model was retrained on the full training set and evaluated on the held-out test set.

For a recent and representative prior-art baseline, we implemented the approach introduced in BLEACH [29], which integrates static signature information with inter-frame spacing (IFS) timing features. Originally, this method was developed to associate probe requests across different time frames and mixed-device streams. However, because our dataset is organized per-device rather than as continuous traffic, we adapted the system for device-type classification. We retained its core architecture—feature extraction, trail segmentation, and logistic regression to ensure a fair comparison. Each device’s PCAP was divided into one-second trails, discarding any segment with fewer than four frames, consistent with the original design. From the valid trails, we extracted IE presence and the mean IFS, forming feature vectors that were used to train a logistic regression model with the default hyperparameters of the original implementation.

We also implemented a neural network baseline following the approach described in [23], which was specifically designed for identifying device types from probe requests. This method addresses the variability in IEs by encoding each frame as a 200-byte binary string, padding shorter ones as needed. Each byte was grouped into 4-bit segments, yielding a 400-dimensional vector of averaged hexadecimal values per frame. The resulting vectors were passed to a fully connected neural network comprising three hidden layers, each with 128 neurons, and ReLU activations. Dropout layers were added between hidden layers to mitigate overfitting. Training was conducted over 250 epochs using stochastic gradient descent (SGD) with categorical cross-entropy loss. To prevent bias toward overrepresented device types, common in real-world datasets, we imposed an upper cap of 500 probe requests per device during training. This ensured fair training conditions and mitigated class imbalance. Our method, by contrast, does not depend on large sample sizes and remains effective with only a single example per device (see Section V-E).

All models were evaluated using the F1 score, which balances precision and recall and is well-suited for imbalanced datasets. We report F1 scores averaged over ten runs, each using different train/test splits. Table VIII summarizes the results across all methods. Our approach consistently achieved an average macro-F1 score of 99%, with corresponding precision and recall values of 99% and 98%, respectively. These results demonstrate the high accuracy and robustness of our method, especially in contrast to baselines that require more data or explicit tuning. A deeper comparison follows in the next section.

B. Performance Results

TABLE VIII: Macro-F1 scores for all evaluated methods

Category	Method	F1 Score (%)
Decision-Based	Decision Tree	88.4
	Random Forest	89.2
	XGBoost	88.0
BLEACH Baseline	BLEACH [29]	77.0
Neural Network	Gu et al. [23]	90.0
Proposed Method	Signature Method	99.0

Among tree-based classifiers, Random Forest scored highest at 89.2%, followed by Decision Trees (88.4%) and XGBoost (88.0%). Though efficient and interpretable, they struggled to distinguish devices with near-identical probe signatures. iPhone 13 and iPhone 14 were often confused because their supported IEs differed only slightly. The key distinction was the differing occurrence probability of the extended attribute. The cellular and wireless specs of iPhone 13 Pro and 14 Pro are also nearly identical [5], [6]. While distinct devices, their main differences stem from hardware and features, not Wi-Fi. Similarly, Google Pixel S7 and Pixel S6 were often misclassified due to overlapping feature sets. Token collisions during hashing diluted small device distinctions. Devices with limited training data—e.g., Lenovo ThinkPad X13 Gen 1 or Meta Lens 2—performed poorly, underscoring the need for sufficient examples to form effective decision boundaries.

Our proposed method consistently achieved 99% F1 across all device classes. By weighting rare attributes according to observed frequency, our approach remains sensitive to subtle variations in probe signatures. This allows accurate distinction of closely related devices and provides inherent robustness against class imbalance. Consequently, it outperforms all three decision-based classifiers, especially for underrepresented or low-sample device types.

BLEACH achieved a lower F1 score of 77.0%. Like decision-based models, it struggled to differentiate devices with highly similar configurations, such as iPhone XR and XS, Pixel 6a and 7, Pixel 2 and 3, and various Xiaomi models. This stems from its fixed IE-based signature design, which uses only a single representative value per information element, discarding potentially important intra-IE variability. Although it incorporates inter-frame spacing (IFS) to capture temporal features, the added timing context provided limited differentiation. Devices built on the same chipset family or released in the same generation often exhibit similar IFS patterns, reducing the discriminative power of timing features. The coarse-grained signature and weak temporal cues significantly limit classification accuracy on fine-grained device types.

The neural network baseline by Gu et al. [23] achieved 90.0% F1, with corresponding precision and recall of 91% and 90%, respectively. While this learning-based approach performed better than BLEACH and decision-based classifiers, it still trailed our method. Figure 7 shows over 90% of devices achieved F1 scores above 90% using our method, compared to fewer than 70% with the neural network.

Figure 8 presents confusion matrices for both our method and the neural network. The diagonal concentration in our matrix indicates highly accurate predictions across nearly all device classes. The neural network shows a more dispersed error pattern. Notable misclassifications in our system include Meta Lens 2 (2 out of 9 samples), SolarEdge solar system (7 out of 30, commonly confused with Acton Air Neo controllers), and Google Pixel 7 (2 out of 7, misclassified as Pixel 6a). Despite these cases, our method demonstrates substantially higher overall accuracy and consistency.

Given that the neural network baseline achieved the best performance among all traditional baselines, it serves as the reference point for subsequent comparative evaluations.

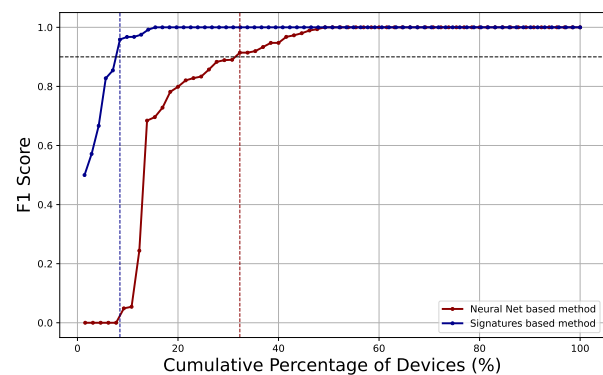


Fig. 7: F1 distribution of device type identification

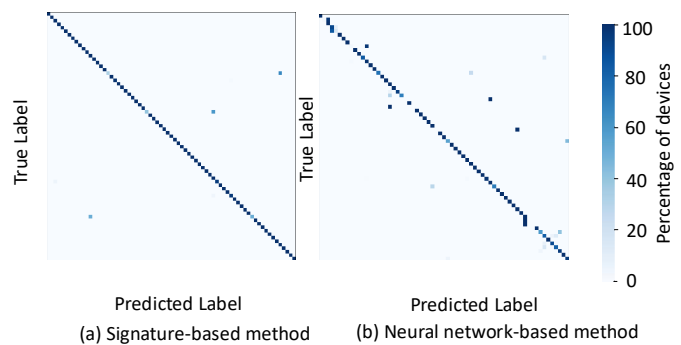


Fig. 8: Confusion matrices showing the % of device types

C. Robustness of Signatures

TABLE IX: Device types with the highest attribute combinations

Device	Total probe request	IE combinations	F1 score
One Plus Nord	500	55	99%
Xiaomi Redmi 4	500	24	95%
Samsung Galaxy s22	58	22	100%
Samsung Galaxy A73	109	13	100%
Google Pixel 5	82	4	100%
Apple iPhone XS	500	3	100%
Macbook Air M1	29	3	100%

1) *Variation in IE*: As previously noted (cf. Section III-B3), the IE within the probe requests of certain devices exhibit variations. Table IX presents the device with the highest number of unique IE combinations. These combinations signify alterations in IE values or changes in a given device's overall set of IEs. For instance, the Apple iPhone XS might showcase 3 IEs in one probe request and 4 IEs in subsequent probe frames. Additionally, the specific values associated with the IEs could differ across certain probe requests for that device. In the overall dataset, 14% of devices exhibit 3 combinations or more, 32% of devices exhibit two combinations each, and 54% maintain consistent IE attributes and values. The F1 score presented in Table IX is the average of 10 runs per device, showing high scores regardless of IE variations. This highlights our method's robustness in capturing IE changes accurately without compromising performance.

2) *Modes of operation*: To assess the reliability of our method across various device modes, we utilized the dataset sourced from [30], which encompasses data gathered in four distinct device modes: A (active), S (static), PA (power-saving while active), and PS (power-saving while static).

We generated signatures for each mode per device and conducted comparative analyses to discern discrepancies. We observed no significant disparities in signatures between normal usage and power-saving modes, i.e., between mode A and PA and between mode S and PS. However, we observed minor variations between modes A and S and PA and PS for four devices. We calculated the accuracies of signatures created for each mode by evaluating signatures generated for one mode and testing them against the other three modes. We perform this experiment for each mode repeatedly. The results, shown in Table X, demonstrate consistent accuracies across all modes, highlighting the robustness of our approach. For instance, from mode A to mode S, we have an accuracy of 99%. The minimum accuracy we got is 97% between PS and PA. In other words, our method remains resilient to variations in signatures, ensuring consistent results.

TABLE X: Accuracy results for mode comparison

Mode	A	PA	S	PS
A	1.000	0.999	0.990	0.987
PA	0.998	1.000	0.991	0.985
S	0.984	0.971	0.993	0.976
PS	0.981	0.970	0.988	0.994

To determine whether device signatures remain stable over time, we analyzed a subset of our data by examining signatures across different time intervals, device updates, and operating system changes.

- Dell Latitude 5460: Monitored monthly over six months while dual-booted with Windows and Linux. Despite multiple Windows updates, the device signature remained consistent.
- Samsung Tab S8: Tested monthly over six months. The device underwent two minor updates to Android 13 and a major update from Android 13 to 14, yet the signature remained stable.
- Google Pixel 6a: Monitored monthly over six months. The device received minor updates to Android 14 and a major update from Android 14 to 15, with no changes in its signature.
- Apple iPhone 13: Analyzed over three months with monthly signature generation. The device underwent a major update from iOS 15 to iOS 16, and its signature remained unchanged.

These results demonstrate the temporal robustness of our device signatures across software updates and operating system changes. This stability stems from signatures reflecting the underlying Wi-Fi capabilities of the network interface card (NIC) and its firmware. Because NIC hardware is fixed per model, firmware updates rarely change fundamental behavior or Information Element (IE) patterns. The signature remains tied to these hardware properties unless the NIC is physically replaced—a rare event.

Our probabilistic approach adds resilience to minor signature changes. If IE patterns shift after major updates and alter the signature, the correct device typically still appears within the top-k matches via probabilistic ranking. For example, even if a firmware update modified the Google Pixel 6a's signature, the device's inherent hardware would be unchanged, and it would still rank among top candidates. While substantial behavioral changes may require retraining, future work could explore adaptive learning to support dynamically evolving signatures.

However, our evaluation covered only four device models over 3–6 months, spanning a limited set of OS and firmware updates. In this small cohort, all extracted signatures remained stable, but sample size and diversity are insufficient for broader generalization. These findings should therefore be viewed as preliminary evidence of stability rather than definitive proof. Larger longitudinal studies are needed to quantify population-level robustness, for example, estimating per-device confidence intervals for match rates across time, firmware versions, and hardware variants.

D. Adeptness of Signatures

To assess signature adeptness, we evaluated how the F1 score changed as the signature was updated from observed data. We used a single probe request in the training set to construct each device's signature, tested on remaining frames, and measured both overall and per-device F1. We then increased the number of training probes in steps of five, repeating the measurements.

Our hypothesis is that a larger training set exposes the signature to a broader mix of IE combinations per device; the signature adapts, improving F1. Our method models the probability of devices exhibiting IE combinations. With more data, it better learns these probabilities, refining similarity between devices. We ran five trials per training size and averaged F1. Table XI shows a clear rise as probes per device increase from 1 to 15, peaking at 97.8

We also computed per-device F1. Fig. 10 plots the cumulative percentage of devices reaching specific F1 levels for four training sizes. As training probes rise from 5 to 15, the share above 90% climbs from 74% to 97%. In summary, F1 consistently increases with more training points, demonstrating our signature-based approach's adeptness. Notably, even a single probe request yields strong performance (92.8% F1), explored further in the next section.

TABLE XI: Average F1 scores with increased training data

	Number of probe requests per device in the training set			
	1	5	10	15
F1 score	92.8 ± 1.4	95.2 ± 0.3	97.4 ± 0.5	97.8 ± 0.6

E. Training data requirement comparison

We explore the minimum number of probe frames needed for each method to identify a single device type, i.e., the minimum training data required per device type. Frames were randomly selected for training, with the rest used for testing.

We began with one frame per device type and incrementally increased to 10 frames. This process was repeated five times for each number of frames, and averages were calculated from the results.

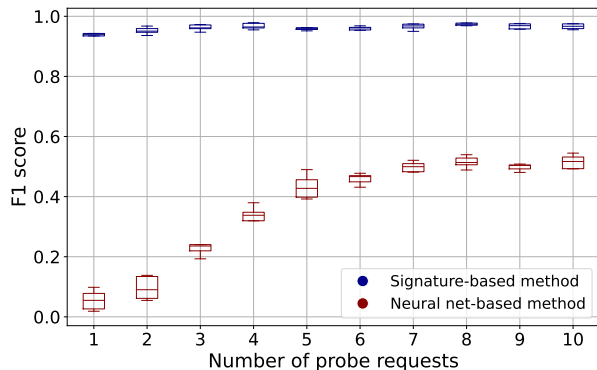


Fig. 9: F1 score for min. number of traces per device

Fig. 9 illustrates the F1 scores for our method and the neural network-based approach trained with 1 to 10 traces per device type. Our method achieved over 92% F1 score with just one trace per device, compared to the neural network's F1 score remaining below 10%. These results highlight our method's effectiveness with minimal training data, showing that even a single trace per device is sufficient for robust representation and comparison of extracted information.

The disparity in results can be attributed to the distinctions in approach between the neural network and our method. Our method differs from the neural network-based method in two key aspects. Firstly, our approach assigns distinct weights to individual IEs within the probe frames for each device type. Secondly, we directly address the inherent variability of IEs within the probe request for a specific device type, calculating the probability of seeing a different combination of IE for each device type. In contrast, the NN generates a vectorised representation of IEs and discerns differences between the vectors of various device types. As a result, it may overlook the significance of each IE for a specific device type, especially when a device type can have multiple IE combinations.

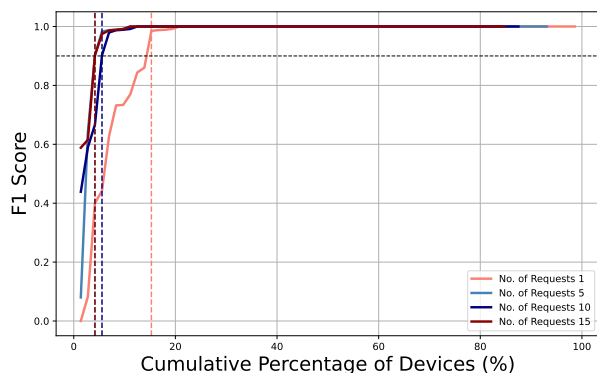


Fig. 10: F1 scores as the number of probe requests increases

F. Unknown-Device Detection

To evaluate the system's ability to flag previously unseen devices as "unknown," we performed a series of hold-out

experiments. In each run, we randomly excluded a certain percentage h of the 73 device types during signature creation (e.g., 10%, 15%, 20%, etc.), then used the remaining signatures as the verification base. This random exclusion was repeated across five independent runs to prevent selection bias. For each run, we computed two metrics:

- **Detection rate** (held-out $\rightarrow$ unknown): the fraction of truly unseen devices correctly flagged as unknown.
- **False-alarm rate** (known $\rightarrow$ unknown): the fraction of known devices erroneously flagged as unknown.

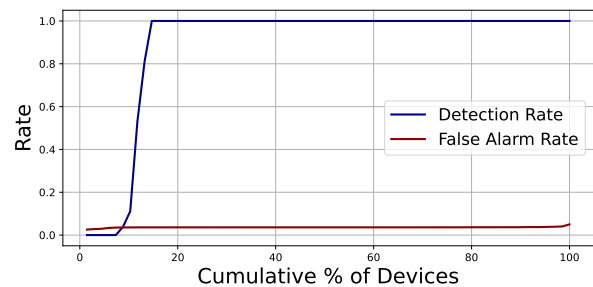


Fig. 11: Detection and False-Alarm Rates by Hold-Out Size

Figure 11 shows the empirical cumulative distribution functions (CDFs) of detection and false-alarm rates across all per-device hold-out experiments, where each of the 73 devices is withheld in turn and tested against signatures from the remaining 72 devices. The detection-rate CDF climbs steeply toward 1.0, indicating that most devices are reliably detected as unknown. A few devices appear as "knees" in the curve, reflecting confusion with closely related models. For instance, Apple iPhone 13 Pro and 14 Pro signatures were often misclassified as each other, rather than flagged as unknown. Similar confusion occurred between TP-Link Smart Plug and Smart Globe, and between Google Pixel 6 and 6a, which led to near-zero detection rates for these pairs. Additionally, some Apple iPhone 11 traces were misclassified as iPhone 12. This pattern indicates that while the system reliably detects unknown devices, it sometimes mislabels a held-out device as a closely related known device instead of "unknown."

We examined robustness at six hold-out levels by excluding $h \in \{2, 5, 10, 14, 15, 18\}$, corresponding to approximately 3%, 7%, 14%, 19%, 21%, and 25% of the 73 devices. For each h , five trials were performed using different random subsets of excluded device types. We recorded the detection and false-alarm rates for each trial, reporting the mean and standard deviation in Table XII.

TABLE XII: Hold-out performance results for detection rate and false alarm rate

Hold-out Count (h)	Detection Rate	False Alarm Rate
2	1.000 ± 0.000	0.039 ± 0.006
5	0.958 ± 0.058	0.035 ± 0.004
10	0.859 ± 0.113	0.028 ± 0.004
14	0.979 ± 0.020	0.034 ± 0.007
15	0.899 ± 0.121	0.031 ± 0.010
18	0.875 ± 0.100	0.037 ± 0.010

As shown, the false-alarm rate remained consistently low across all hold-out sizes, never exceeding 0.04 on average.

Minor fluctuations were observed, particularly at $h = 10$ and $h = 14$, but variability remained low.

Detection performance showed greater variation, reflecting the influence of specific device types in the hold-out set. Despite this, detection rates were generally high: over 0.95 for smaller hold-outs, and above 0.85 on average for larger ones. Even at the most challenging setting ($h = 18$), the mean detection rate stayed above 0.87, with the lowest individual trial remaining over 0.80. These results highlight the strong generalization ability of the proposed method. It maintains high detection accuracy and stable false-alarm rates, even when a large portion of devices is excluded during training. Performance drops were primarily due to device signature similarities, but remained within acceptable bounds.

VI. PILOT STUDY

To evaluate real-world applicability, this pilot tests our method's robustness in uncontrolled environments. By clustering the similarity scores calculated by our method, we identify patterns and natural relationships between devices and behaviors. These insights enable us to refine the method, ensuring better performance in unpredictable, live deployments.

A. Data Collection and Experiment Setup

We collected data from multiple university campus locations, chosen to represent diverse environmental conditions and device densities. These included high-density, low-mobility environments (e.g., the library), moderate-density, high-mobility areas (e.g., the computer science building), and low-density outdoor spaces (e.g., the campus courtyard). Using the same setup as outlined in Section IV-A, we employed three ALFA AWUS036NHA external WiFi antennas connected to either a Dell XPS 15 laptop running Linux or a Raspberry Pi running Raspbian OS with custom Python scripts for data collection. Each location was surveyed for 30 minutes at varying times across multiple days to capture temporal diversity in the data, accounting for fluctuations in device presence and activity levels.

Following the data preprocessing steps outlined in Section IV-B, we implemented a filtering step to address duplicate entries caused by repeated appearances of the same MAC address. In real-world scenarios, certain devices emit significantly more probe requests than others, resulting in an imbalance where these devices are disproportionately represented in the dataset. We retained only a single data point for each unique MAC address to mitigate this. This ensured uniform representation of devices, reducing redundancy and enabling an equitable evaluation of our methodology across all devices. With the dataset balanced, we constructed an $N \times N$ similarity matrix, where N denotes the number of unique data points. Each entry in the matrix represented the match probability between a pair of data points, calculated using our matching algorithm. This involved performing exhaustive pairwise comparisons, resulting in a dense matrix that captures the degree of similarity between all possible data point combinations. The similarity matrix highlighted dataset relationships, with high probabilities indicating strong similarities and low probabilities showing distinct differences.

B. Cluster Analysis

We applied hierarchical clustering to explore device grouping patterns based on similarity scores, taking advantage of its flexibility for variable device distributions. Hierarchical clustering offered two key benefits: it did not require predefined clusters, accommodating the variability of real-world data, and its tree-like structure allowed for multiresolution analysis, highlighting patterns in device type and model clustering. The similarity matrix, which calculated distances inversely proportional to similarity scores, was used as the input for clustering. The heatmap in Fig. 12 visualized the pairwise similarity scores, with rows and columns representing individual data points and the color intensity indicating the degree of similarity between them. Together, the dendrogram and the heatmap provided a comprehensive view of the clustering structure, revealing whether devices naturally grouped according to model or type. When examining the heatmap, three distinct clusters emerged: Apple devices, Android devices, and unknown devices. These groups were visually separable and exhibited consistent intra-group similarity. Based on the relative area covered by each cluster in the heatmap, which corresponds to the proportion of devices within each group out of the total 937 devices analyzed—the unknown device cluster represented 41.0% of the total, Apple devices accounted for 27.8%, and the remaining 31.2% corresponded to Android devices. These proportions illustrate how the clustering effectively captured the dominant groupings in the dataset, highlighting strong structural distinctions between device categories.

Within the Android cluster, we can observe smaller sub-clusters corresponding to specific brands like Xiaomi, Google, and Samsung. However, these sub-clusters are not entirely distinct—there is some overlap between them. This overlap occurs because these brands often use similar WiFi chipsets sourced from suppliers like Broadcom, Qualcomm, and MediaTek. Since these chipsets share identical architectures, the resulting probe request signatures also exhibit similarities. Despite this overlap, nuanced combinations of features within the probe requests allow for differentiation between these brands (inter-group) and within the devices of a single brand (intra-group). Key features enabling this differentiation include capabilities such as extended throughput and high throughput, which highlight both group-level and individual device distinctions. In contrast, Apple devices form a distinct cluster separate from the Android brands. Apple typically uses proprietary or customized versions of off-the-shelf chips, often from Broadcom, but with firmware or hardware optimizations that produce unique probe request characteristics. This makes Apple devices easily identifiable and differentiated from Android devices, particularly due to their distinct capabilities.

Interestingly, the Apple cluster tends to be larger than the combined clusters of other device groups. At first glance, one might assume this simply reflects a higher number of Apple devices. However, based on data collected in both our controlled laboratory experiments and wild data settings, we hypothesize that the underlying cause is more nuanced and relates to differences in how Apple implements MAC address randomization compared to other vendors. When generating

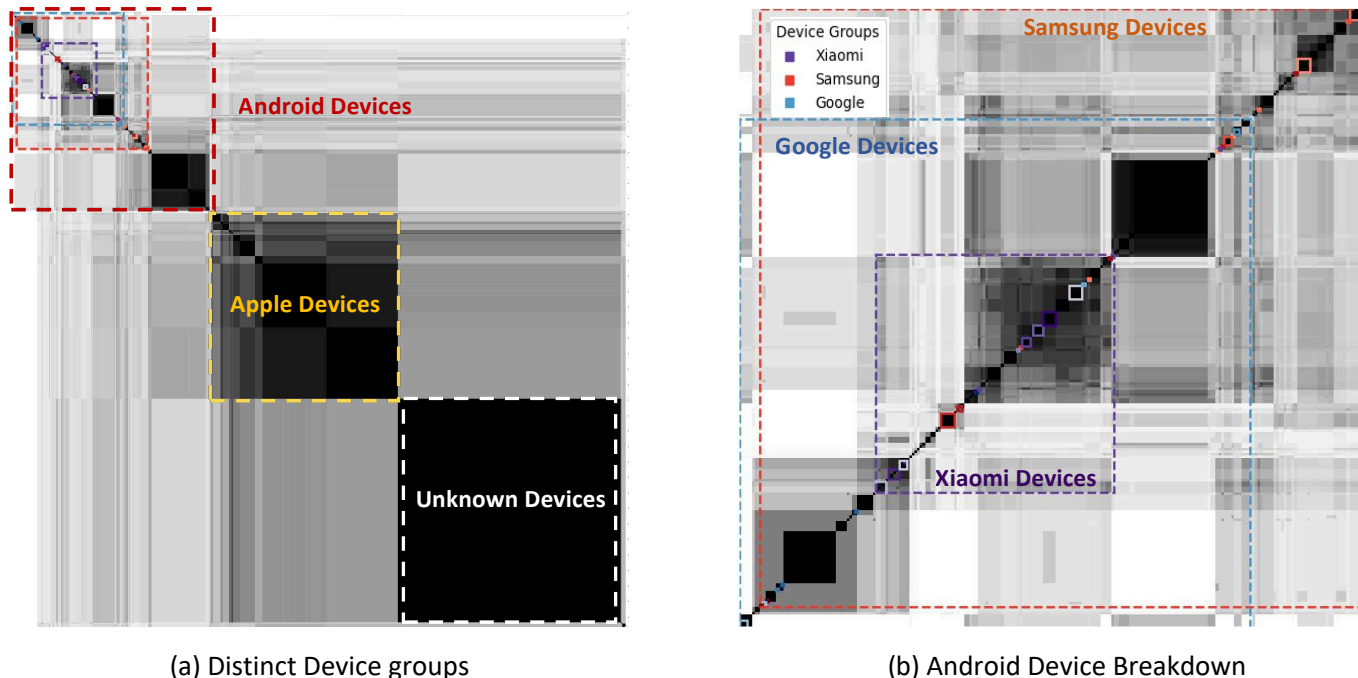


Fig. 12: Cluster Map based on Device Similarity Scores

clusters, each MAC address was ensured to be unique to avoid duplicates, and Apple’s randomization behavior appears to play a key role. Specifically, our observations indicate that this difference arises primarily in wildcard (undirected) probe requests. Apple devices use a different randomized MAC address for each probe burst in wildcard Probe Requests, while directed Probe Requests (targeting known SSIDs) often reuse the same MAC address. In contrast, Android devices use the same MAC address for both directed and undirected probe requests for a given SSID. For instance, during a typical two-minute experiment involving ten probe bursts, an Apple device may generate ten distinct MAC addresses, whereas an Android or Google device would likely reuse one. This behavior produces a larger number of unique MAC entries for Apple devices, resulting in more extensive clustering. We emphasize that this represents a single observation from our dataset and not a confirmed finding. Further analysis and testing are required to verify this behavior and fully understand the underlying mechanisms. We therefore present it solely as an observed pattern that may help explain the clustering behavior of Apple devices.

We emphasize this is an empirical pattern in our dataset, not a general claim. Comprehensive validation across device models and software versions is beyond this paper’s scope. Establishing causality, assessing prevalence, and reconciling the behavior with per-connection MAC policies would require targeted experiments. We thus present it as future work and a plausible explanation for the observed clustering.

Next, we discuss the scalability of our matching approach, which compares each *target* signature against every entry in the *verification database*, by examining its time and space

complexity. This brute force method performs roughly $\frac{n(n-1)}{2}$ comparisons, each costing M , where n is the number of signatures: $T(n) = O(n^2M)$. Memory usage also grows quadratically due to an $n \times n$ similarity matrix: $S(n) = O(n^2)$. Since the number of signatures grows linearly with packets, total runtime shows a quadratic trend, consistent with the underlying complexity. In about 95% of cases, a probe yields a perfect match. This allows early termination, typically after a single comparison, with cost $T_{\text{perfect}} = O(M)$. Combining perfect and non perfect cases, the expected time is $T_{\text{mixed}}(n) = O(nM)$, reflecting that most probes end early while some require a full scan. Thus the average case drops from quadratic to linear in n , improving runtime for realistic workloads.

TABLE XIII: Time complexity under different matching scenarios

Scenario	Comparisons	Time Complexity
Brute-force baseline	all pairs	$O(n^2 \cdot M)$
Perfect matches only	single comparison	$O(M)$
Mixed-case (95%)	mostly early exit	$O(n \cdot M)$

Although early termination improves average performance, the worst case scales poorly. To address this, future work could explore sublinear time indexing by organizing the verification database as a balanced Merkle tree: each leaf stores a device signature hash, and each internal node stores the hash of its concatenated children. During lookup, the probe’s hash is compared hierarchically with internal nodes, pruning large candidate subsets without exhaustive pairwise comparisons. This reduces worst-case verification time.

VII. REAL-TIME DEVICE IDENTIFICATION: DASHBOARD & APPLICATIONS

Building on our clustering experiment, this section operationalizes the device type identifier from Sections III–V. We developed a Grafana-based¹ dashboard as a presentation layer for the classifier's outputs. This tool bridges clustering analysis and practical application, providing an intuitive interface for real-time device monitoring. The code is available on GitHub.

1) *Dashboard Overview*: The Landing Page Fig. 13(a) is the central hub for real time monitoring and probe request categorization. It includes the Live Feed Display ①, which shows a dynamic view of incoming probes by device type, demonstrating processing as data arrives. The Device Breakdown Panel ② lets users inspect each detected device with a detailed breakdown from the database. Probability Analysis ③ explains how closely each probe matches specific devices, with probabilities clarifying classification results. The Probe Calculation Summary ④ outlines the steps and calculations behind probe categorization, improving interpretability.

The Interactive Map Fig. 13(b) visualizes probe data from multiple monitored sites with tools to explore the data. The Site Specific Device Breakdown lets users select a site and view device types detected there. Customizable Filtering Options refine the view by manufacturer, model, or other criteria. Within these filters, Vendor and Device Type Analysis shows the distribution of specific vendors or device types at a site. Finally, Cross Site Comparisons allow users to track devices across sites, using data such as MAC addresses to identify patterns in device mobility or reuse.

The dashboard was developed with real-world applications in mind, aligning with our use cases:

A. Device Type Identification and Its Role in Urban Planning

The dashboard enables urban planners to assess neighborhood technology landscapes by identifying device types such as IoT devices, smartphones, laptops, and smart home technologies. An interactive map shows device distribution in real time, revealing actionable differences that guide targeted infrastructure investments and resource allocation. Areas with high device density, where IoT systems and smart homes are common, indicate communities with stronger connectivity needs. Planners can prioritize these zones for advanced networks and smart city solutions such as traffic monitoring, sensor based waste management, or IoT enabled safety systems. The dashboard highlights these concentrations in real time, supporting timely, evidence based decisions.

Areas with lower device density, characterized by basic mobile phones or laptops, may not require complex infrastructure but still benefit from improved mobile coverage, public Wi-Fi, and digital inclusion initiatives. Recognizing these patterns helps planners allocate resources efficiently and bridge gaps in connectivity, promoting balanced urban development.

Socioeconomic context further enriches this analysis. Wealthier areas may contain more advanced technologies, while device data reveal disparities and highlight underserved

communities. This insight enables targeted initiatives such as digital literacy training or affordable broadband programs. By integrating socioeconomic and technological data, the dashboard ensures that both digital and social priorities inform planning decisions.

While traditional proxies such as median household income, population density, and education levels offer useful indicators for urban planning, the dashboard complements them by providing direct, empirical insights into device type distribution—including IoT and infrastructure devices—that these proxies only indirectly reflect. For example, median income data from government surveys may indicate general patterns in technology adoption, but it does not audit the actual infrastructure or devices present. Our approach addresses this by enabling real-time audits of device presence, capturing not only personal devices but also IoT sensors, infrastructure systems, and specialized equipment such as CCTV cameras and smart meters. It further provides planners and agencies with actionable intelligence for infrastructure investment, network capacity planning, and security monitoring based on real device activity rather than inferred trends. Finally, it offers spatially and temporally granular data on device types and mixes within specific environments, revealing behavioral and technological patterns beyond the reach of income metrics.

The dashboard's real-time monitoring makes it a dynamic tool for adaptive urban planning. By continuously tracking technological engagement across neighborhoods, it helps planners direct resources according to current needs. Integrating device data with socioeconomic indicators supports smarter investment in high-tech areas while addressing connectivity gaps elsewhere, promoting equitable and data-driven urban development.

B. Real-Time ISP Network Optimization and Planning

By identifying the types of devices actively using the network, ISPs can gain insights into network traffic needs and optimize service delivery. The dashboard categorizes devices such as IoT devices, smartphones, smart TVs, gaming consoles, and laptops, displaying this information in real-time. With this data, ISPs can prioritize bandwidth allocation based on device requirements. For example, areas with a higher concentration of smart TVs or gaming consoles, which demand high, consistent bandwidth for streaming or gaming, can receive priority for higher speeds. Meanwhile, regions with IoT devices, which generate high volumes of low-bandwidth traffic, can have bandwidth allocated accordingly, reducing congestion. Additionally, the dashboard's insights into device type distribution enable ISPs to predict future network demands. If there's an increase in smart home devices in a neighborhood, ISPs can anticipate the need for additional bandwidth. By monitoring these trends, ISPs can proactively adjust their infrastructure to meet future demands, ensuring efficient use of resources and improved customer satisfaction. This capability could also extend to a more granular level, allowing for tailored service offerings based on household device usage patterns. For example, households with multiple high-demand devices could be allocated additional bandwidth,

¹<https://grafana.com/>

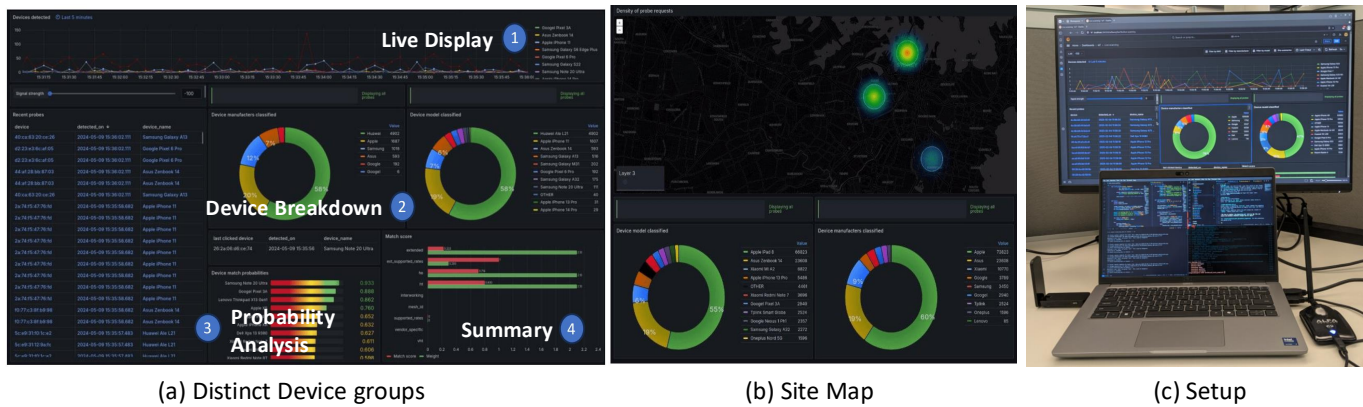


Fig. 13: Dashboard for Real-Time Monitoring

while those with fewer devices might be assigned a more stable, lower-bandwidth service. This level of detail would enable ISPs to optimize both regional and individual customer experiences, ensuring resources are allocated efficiently across their network. Ultimately, the dashboard serves as a powerful real-time monitoring tool, empowering ISPs to make data-driven decisions, improve service delivery, and ensure more efficient use of network resources.

VIII. DISCUSSION

Identifying device models in real-world Wi-Fi settings is challenging as OUI-based inference is often unreliable; many devices use randomized MAC addresses, rendering the OUI uninformative. We address this via a passive approach that analyzes management frames broadcast during normal operation. By leveraging hardware-influenced variations in a minimal set of Information Elements to form device-specific signatures, our method requires no active network interaction. Evaluated on 73 device types, our approach attains a 99% F1 score, substantially exceeding comparable benchmarks (typically $\sim 90\%$) and demonstrating high-accuracy identification under realistic conditions.

Passive Data Collection: Unlike active approaches that transmit probes or engage directly with network devices to solicit responses, our method is strictly passive. It listens to the ambient wireless environment and captures management frames that are already broadcast by devices, primarily probe requests. This strategy avoids any form of signal injection or direct querying, thus minimizing interference and exposure. While passivity reduces the intrusiveness of data collection, it does not entirely eliminate privacy risks. Device broadcasts, though necessary for network discovery, can still reveal implicit identifiers or timing patterns that facilitate potential re-identification. Our contribution is therefore not to claim absolute privacy, but to design a system that operates within the minimal exposure footprint of what devices already share voluntarily during normal operation.

Privacy and Ethics: We acknowledge that passive Wi-Fi sensing presents both opportunities and risks. Many mobile device manufacturers obfuscate or randomize broadcast identifiers precisely to prevent long-term device tracking. Our system is designed in alignment with these privacy-enhancing trends: As

mentioned before, the operation is passive and does not require probing or interacting with the device. Secondly, we are adhering to data minimization, collecting only absolutely necessary information. In practice, this means we extract only a subset of information from each captured frame, those strictly required for the task of device-type classification. Other metadata that could be used to track, localize, or identify individual devices, such as MAC addresses, timestamps, RSSI, or payload content, is immediately discarded. This approach ensures that persistent identifiers are not retained, and each observation is treated as a standalone, unlinkable event. The system never stores raw frames or reconstructs session-level behavior, aligning with privacy-by-design principles [10], [38], [39].

The output of our system consists only of aggregated device statistics. For example, it may report how many observed devices were iOS or Android, give distributions such as counts of Apple, Samsung, and Huawei devices, or provide model level breakdowns like iPhone 14 or Galaxy S22. We do not attempt to determine the number of unique devices of any type (e.g., “13 distinct iPhone 14s”), and we do not associate any data with specific users. While our design makes reidentification highly impractical, there is a theoretical possibility of linking repeated observations through side channels such as timing. For instance, repeated probe requests of the same device type within a narrow window could suggest recurrence. However, without persistent identifiers, such inferences are limited in scope and reliability. Future work may explore stricter temporal aggregation or lightweight obfuscation to further reduce even these low risk scenarios.

Similar WiFi monitoring systems producing aggregate data are used for crowd estimation and people counting. During the COVID-19 pandemic, researchers in Madeira, Portugal used passive WiFi sensing to monitor crowd densities in public spaces without identifying individuals [19]. Commercial tools FootfallCam² and Xovis³ are deployed in retail, airports, and public venues to report visitor counts and flow patterns. This broad use shows acceptance of non-identifiable WiFi monitoring for aggregate insights, yet ethical concerns about user awareness remain; deployments should provide clear signage, transparent data policies, and oversight to maintain

²<https://www.footfallcam.com/>

³<https://www.xovis.com/>

public trust and emphasize aggregate analysis, not individual surveillance. *To summarize, our design follows best practices for safeguarding user privacy in data collection and handling, and we recommend clear signage for public deployments.*

Future work A current limitation is that our experiments used only 2.4 GHz traces and did not evaluate 5 GHz. Although the method matches on 802.11 Information Elements that are present across bands, this leaves a gap in experimental validation. Future work will address this by extending the study to 5 GHz and to settings where both bands are present. In deployment, a dual band device would maintain separate signatures for 2.4 GHz and 5 GHz, and the matcher would use the signature for the band that is observed. Another direction is to combine Layer 1 features such as signal strength, phase, and frequency offset with our signature to improve accuracy and robustness under varying network conditions and device behaviors [11], [25]. Finally, we will investigate scaling the system for large deployments, including techniques such as Merkle trees and perceptual hashing to support efficient storage, updates, and lookup, as discussed in Section VI. Finally, while we model within-device variability in wildcard probe request IEs, our dataset does not explicitly separate observations by device association status. As a result, we do not attribute observed IE variability to specific connectivity states. A controlled study that isolates unassociated/associated conditions is an important direction for future work. We also note that because our similarity score is driven by empirical occurrence probabilities of probe-request IE values, closely related device models may exhibit partially overlapping signatures, leading to occasional misclassification (e.g., iPhone 13 Pro vs. 14 Pro). Our calibration-based rarity weighting and storing observed per-device IE variations reduce this ambiguity, but cannot fully eliminate it when devices advertise near-identical Wi-Fi capability IEs. These efforts aim to refine deployment strategies and broaden applicability across diverse public environments.

IX. CONCLUSION

In conclusion, this paper presents a signature based approach for passive WiFi device type identification, validated by experiments in a university setting. Our method uses 73 popular WiFi devices, the largest dataset in this domain to date. It complements prior work by better distinguishing closely related variants and improving tracking accuracy. Our approach achieves a 99% F1, significantly outperforming a comparable neural network based method at 90%. It also maintains strong performance ($F1 \geq 92\%$) with minimal training data, while the neural model fails under the same conditions. These results highlight the technique's robustness, simplicity, and scalability. Although expanding the device database may add ambiguity, this can be mitigated by augmenting the signature with lightweight features. Overall, the method is a practical and interpretable alternative to black box models, enabling accurate passive identification in real world settings and laying groundwork for future cross layer and temporal identification.

X. ACKNOWLEDGEMENT

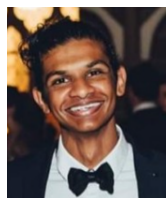
This research is funded by the Department of Defense Australia [Agreement No: 11384], between UNSW and the

Department of Defence, and National Intelligence and Security Discovery Research Grant (NI220100111), Australia.

REFERENCES

- [1] "TShark - the network protocol analyzer," <https://www.wireshark.org/docs/man-pages/tshark.html>.
- [2] "IEEE Standard for Information technology- Local and metropolitan area networks- Specific requirements- Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 5: Enhancements for Higher Throughput," *IEEE Std 802.11n-2009 (Amendment to IEEE Std 802.11-2007 as amended by IEEE Std 802.11k-2008, IEEE Std 802.11r-2008, IEEE Std 802.11y-2008, and IEEE Std 802.11w-2009)*, pp. 1-565, 2009.
- [3] "IEEE Standard for Information technology- Telecommunications and information exchange between systems Local and metropolitan area networks- Specific requirements-Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications- Amendment 4: Enhancements for Very High Throughput for Operation in Bands below 6 GHz," *IEEE Std 802.11ac-2013 (Amendment to IEEE Std 802.11-2012, as amended by IEEE Std 802.11ae-2012, IEEE Std 802.11aa-2012, and IEEE Std 802.11ad-2012)*, pp. 1-425, 2013.
- [4] S. Aneja, N. Aneja, and M. S. Islam, "Iot device fingerprint using deep learning," in *2018 IEEE international conference on internet of things and intelligence system (IOTAIS)*. IEEE, 2018, pp. 174-179.
- [5] Apple Inc., "iPhone 13 Pro - Technical Specifications," https://support.apple.com/kb/SP852?locale=en_GB, 2022, accessed: February 12, 2024.
- [6] Apple Inc., "iPhone 14 Pro - Technical Specifications," https://support.apple.com/kb/SP875?locale=en_GB, 2023, accessed: February 12, 2024.
- [7] P. Baral, N. Yang, and N. Weng, "Iot device identification using device fingerprint and deep learning," in *Deep Learning and Reinforcement Learning*. IntechOpen, 2023.
- [8] M. V. Barbera, A. Epasto, A. Mei, S. Kosta, V. C. Perta, and J. Stefa, "Crawdad dataset sapienza/probe-requests (v. 2013-09-10)," 2013.
- [9] N. Bogahawatta, Y. S. Karunanayaka, S. Seneviratne, K. Thilakarathna, R. Masood, S. Kanhere, and A. Seneviratne, "Passive Identification of WiFi Devices At-Scale: A Data-Driven Approach," in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*. IEEE, 2024, pp. 1-9.
- [10] A. Cavoukian, "Privacy by design the 7 foundational principles, 2011," *Ontario: Information and Privacy Commissioner of Ontario*, 2011.
- [11] B. Chen, Y. Song, Z. Zhu, S. Gao, J. Wang, and A. Hu, "Authenticating mobile wireless device through per-packet channel state information," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE, 2021, pp. 78-84.
- [12] L. C. Choong, C. C. Yuan, T. C. Pheng, and D. Lee, Ri Seng, "Identifying unique devices through wireless fingerprinting," in *Proceedings of the first ACM conference on Wireless network security*, 2008, pp. 46-55.
- [13] Cisco Inc., "Meraki and MAC Address Randomization," https://documentation.meraki.com/General_Administration/Cross-Platform_Content/Meraki_and_MAC_Address_Randomization.
- [14] M. Cunche, "I know your MAC address: targeted tracking of individual using Wi-Fi," *Journal of Computer Virology and Hacking Techniques*, vol. 10, pp. 219-227, 2014.
- [15] M. Cunche, M. A. Kaafar, and R. Boreli, "I know who you will meet this evening! linking wireless devices using Wi-Fi probe requests," in *2012 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2012, pp. 1-9.
- [16] M. Cunche, M.-A. Kaafar, and R. Boreli, "Linking wireless devices using information contained in Wi-Fi probe requests," *Pervasive and Mobile Computing*, vol. 11, pp. 56-69, 2014.
- [17] L. C. C. Desmond, C. C. Yuan, T. C. Pheng, and R. S. Lee, "Identifying unique devices through wireless fingerprinting," in *Proceedings of the first ACM conference on Wireless network security*, 2008, pp. 46-55.
- [18] A. Di Luzio, A. Mei, and J. Stefa, "Mind your probes: De-anonymization of large crowds through smartphone WiFi probe requests," in *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*. IEEE, 2016, pp. 1-9.
- [19] M. et al., "Monitoring crowd densities using passive WiFi sensing during COVID-19," *Journal of Big Data*, 2022. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00675-3>
- [20] E. Fenske, D. Brown, J. Martin, T. Mayberry, P. Ryan, and E. C. Rye, "Three Years Later: A Study of MAC Address Randomization In Mobile Devices And When It Succeeds," *Proc. Priv. Enhancing Technol.*, vol. 2021, no. 3, pp. 164-181, 2021.

- [21] J. Franklin, D. McCoy, P. Tabriz, V. Neagoe, J. V. Randwyk, and D. Sicker, "Passive data link layer 802.11 wireless device driver fingerprinting," in *USENIX Security Symposium*, vol. 3, no. 2006, 2006, pp. 16–89.
- [22] M. S. Gast, *802.11 Wireless Networks: The Definitive Guide*. O'Reilly Media, 2002.
- [23] X. Gu, W. Wu, X. Gu, Z. Ling, M. Yang, and A. Song, "Probe request based device identification attack and defense," *Sensors*, vol. 20, no. 16, p. 4620, 2020.
- [24] T. He, J. Tan, and S.-H. G. Chan, "Self-Supervised Association of Wi-Fi Probe Requests Under MAC Address Randomization," *IEEE Transactions on Mobile Computing*, 2022.
- [25] J. Hua, H. Sun, Z. Shen, Z. Qian, and S. Zhong, "Accurate and efficient wireless device fingerprinting using channel state information," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 1700–1708.
- [26] G. Z. Khan, E.-C. Park, and R. Gonzalez, "M 3-cast: A novel multicast scheme in multi-channel and multi-rate wifi direct networks for public safety," *IEEE Access*, vol. 5, pp. 17 852–17 868, 2017.
- [27] E. Khorov, A. Kiryanov, A. Lyakhov, and G. Bianchi, "A Tutorial on IEEE 802.11ax High Efficiency WLANs," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 197–216, 2019.
- [28] J. Martin, T. Mayberry, C. Donahue, L. Foppe, L. Brown, C. Riggins, E. C. Rye, and D. Brown, "A study of MAC address randomization in mobile devices and when it fails," *arXiv:1703.02874*, 2017.
- [29] A. K. Mishra, A. C. Viana, and N. Achir, "Bleach: From WiFi probe-request signatures to MAC association," *Ad Hoc Networks*, vol. 164, p. 103623, 2024.
- [30] L. Pintor and L. Atzori, "A dataset of labelled device Wi-Fi probe requests for MAC address de-randomization," *Computer networks*, vol. 205, p. 108783, 2022.
- [31] M. M. Rathore, A. Ahmad, A. Paul, and S. Rho, "Urban planning and building smart cities based on the internet of things using big data analytics," *Computer networks*, vol. 101, pp. 63–80, 2016.
- [32] S. Seneviratne, F. Jiang, M. Cunche, and A. Seneviratne, "SSIDs in the wild: Extracting semantic information from WiFi SSIDs," in *2015 IEEE 40th Conference on Local Computer Networks (LCN)*. IEEE, 2015, pp. 494–497.
- [33] I. C. society, "IEEE Draft Standard for Information Technology - Telecommunications and Information Exchange Between Systems - Local and Metropolitan Area Networks - Specific Requirements - Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," *IEEE P802.11-REVma/D89.0*, 2010.
- [34] J. Tan and S.-H. Gary Chan, "Efficient association of wi-fi probe requests under mac address randomization," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021, pp. 1–10.
- [35] M. Vanhoef, C. Matte, M. Cunche, L. S. Cardoso, and F. Piessens, "Why MAC address randomization is not enough: An analysis of Wi-Fi network discovery mechanisms," in *Proceedings of the 11th ACM on Asia conference on computer and communications security*, 2016, pp. 413–424.
- [36] O. Waltari and J. Kangasharju, "The wireless shark: Identifying wifi devices based on probe fingerprints," in *Proceedings of the First Workshop on Mobile Data*, 2016, pp. 1–6.
- [37] J. Weppner, B. Bischke, and P. Lukowicz, "Monitoring crowd condition in public spaces by tracking mobile consumer devices with wifi interface," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing: Adjunct*, 2016, pp. 1363–1371.
- [38] Q. Xu, R. Zheng, W. Saad, and Z. Han, "Device fingerprinting in wireless networks: Challenges and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 94–104, 2015.
- [39] Z. Yang and K. Järvinen, "Modeling privacy in WiFi fingerprinting indoor localization," in *International Conference on Provable Security*. Springer, 2018, pp. 329–346.
- [40] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Communications of the ACM*, vol. 64, no. 3, pp. 107–115, 2021.



Niruth Bogahwatta completed an undergraduate degree in Advanced Computing and Finance with First Class Honours. Currently pursuing a PhD in Computer Science at the University of Sydney. Previously worked as a research assistant at the University of Sydney and UNSW. Research interests include network security and distributed systems.



Yasiru Senarath Karunanayaka is pursuing a Bachelor of Engineering in Electrical, Electronics, and Communications Engineering at the University of Moratuwa. He works as an Associate Embedded Systems Engineer at Paraqum Technologies. He was also a Research Affiliate at the University of Sydney, where he contributed to research on IoT fingerprinting and data security.



Suranga Seneviratne is an Associate Professor in Security at the University of Sydney. He earned his Ph.D. from UNSW in 2015. His research focuses on privacy and security in mobile systems, AI applications, and behavioral biometrics. Before research, he spent six years in telecommunications network planning and operations. He holds a bachelor's degree from the University of Moratuwa, Sri Lanka.



Kanchana Thilakarathna received a Ph.D. in electrical engineering and telecommunications from the University of New South Wales. He is a researcher at Data61's Networks Group and a joint lecturer at UNSW's School of Computer Science and Engineering. His research includes resource allocation, secure communication, authentication, and privacy in wearable, mobile, and IoT networks. He won the Malcolm Chaikin Prize in 2015.



Rahat Masood is a Lecturer at UNSW's School of Computer Science and Engineering. She earned her Ph.D. from UNSW in collaboration with Data61-CSIRO. Her research focuses on cybersecurity, including privacy-preserving technologies, authentication, critical infrastructure, and network security measurement. Before UNSW, she was a postdoctoral fellow at Data61-CSIRO. She also holds a Master's from NUST, Pakistan.



Salil S. Kanhere (Senior Member, IEEE) earned M.S. and Ph.D. degrees from Drexel University. He is a Professor of computer science and engineering at UNSW Sydney. He serves on IEEE/ACM conference committees. His research spans IoT, blockchain, pervasive computing, cybersecurity, and applied machine learning. He is a Senior ACM Member, Humboldt Fellow, ACM Distinguished Speaker, Editor-in-Chief of *Ad Hoc Networks*, and Associate Editor for several IEEE journals.



Aruna Seneviratne (Senior Member, IEEE) is a foundation professor of telecommunications at UNSW Sydney, holding the Mahanakorn Chair. He has worked at universities in Australia, the UK, and France, and with organizations like Muirhead, Avaya Labs, and Telstra. His research includes physical analytics, real-time intelligent systems, behavioral biometrics, wearable optimization, and IoT verification.



Albert Y. Zomaya is a Professor of High Performance Computing Networking and ARC Professorial Fellow at the University of Sydney, directing the Centre for Distributed and High Performance Computing. Author of seven books and 400+ papers, he is Editor-in-Chief of *IEEE Transactions on Computers* and a fellow of multiple prestigious engineering societies.